

Parser.c - Function-Wise Code Explanation

Table of Contents

1. [Helper Functions](#)
 2. [Tokenization Functions](#)
 3. [Symbol Table Functions](#)
 4. [Parsing Functions](#)
 5. [Main Function](#)
-

1. Helper Functions

1.1 `int isKeyword(char *str)`

Purpose: Determines if a given string is a reserved keyword.

Parameters:

- `str` - String to check

Returns:

- `1` if the string is a keyword
- `0` if the string is not a keyword

Logic:

```
int isKeyword(char *str)
{
    for (int i = 0; i < NUMBER_OF_KEYWORDS; i++)
    {
        if (strcmp(str, keywords[i]) == 0)
        {
            return 1; // Match found
        }
    }
    return 0; // No match
}
```

How it works:

1. Iterates through the `keywords[]` array which contains: `{"int", "print"}`
2. Compares input string with each keyword using `strcmp()`
3. Returns `1` immediately when a match is found
4. Returns `0` if no matches found after checking all keywords

Example:

```
isKeyword("int")    → returns 1 (is a keyword)
isKeyword("print")  → returns 1 (is a keyword)
isKeyword("hello")  → returns 0 (not a keyword)
```

1.2 `int isOperator(char *str)`

Purpose: Checks if a string is a valid operator.

Parameters:

- `str` - String to check

Returns:

- `1` if the string is an operator
- `0` if the string is not an operator

Logic:

```
int isOperator(char *str)
{
    for (int i = 0; i < NUMBER_OF_OPERATORS; i++)
    {
        if (strcmp(str, operators[i]) == 0)
        {
            return 1; // Match found
        }
    }
    return 0; // No match
}
```

How it works:

1. Iterates through the `operators[]` array: `{"+", "="}`
2. Uses `strcmp()` to compare input with each operator
3. Returns `1` on first match
4. Returns `0` if no operator matches

Example:

```
isOperator("+")    → returns 1 (is an operator)
isOperator("=")    → returns 1 (is an operator)
isOperator("-")    → returns 0 (not supported)
```

1.3 `int isSymbol(char *str)`

Purpose: Determines if a string is a valid symbol (delimiter/punctuation).

Parameters:

- `str` - String to check

Returns:

- `1` if the string is a symbol
- `0` if the string is not a symbol

Logic:

```
int isSymbol(char *str)
{
    for (int i = 0; i < NUMBER_OF_SYMBOLS; i++)
    {
        if (strcmp(str, symbols[i]) == 0)
        {
            return 1; // Match found
        }
    }
    return 0; // No match
}
```

How it works:

1. Checks against `symbols[]` array: `{"(", ")", ";"}`
2. Compares using `strcmp()`
3. Returns `1` if match found, `0` otherwise

Example:

```
isSymbol("(")    → returns 1 (is a symbol)
isSymbol(";")    → returns 1 (is a symbol)
isSymbol("{")    → returns 0 (not supported)
```

1.4 `int isIdentifier(char *str)`

Purpose: Validates if a string follows valid identifier naming rules.

Parameters:

- `str` - String to validate

Returns:

- `1` if the string is a valid identifier
- `0` if the string is not a valid identifier

Logic:

```
int isIdentifier(char *str)
{
    // First character must be letter or underscore
    if ((isalpha(str[0]) || str[0] == '_'))
    {
        // Remaining characters must be alphanumeric or underscore
        for (int i = 1; str[i] != '\0'; i++)
        {
            if (!isalnum(str[i]) && str[i] != '_')
            {
                return 0; // Invalid character found
            }
        }
        return 1; // Valid identifier
    }
    return 0; // Invalid first character
}
```

How it works:

1. **First check:** Ensures first character is a letter (a-z, A-Z) or underscore (_)
2. **Loop check:** Validates remaining characters are alphanumeric or underscore
3. Returns **0** if any invalid character found
4. Returns **1** if all rules satisfied

Valid identifier rules:

- Must start with letter or underscore
- Can contain letters, digits, and underscores
- Cannot be a keyword

Examples:

```
isIdentifier("x")           → returns 1 (valid)
isIdentifier("_var")        → returns 1 (valid)
isIdentifier("var123")      → returns 1 (valid)
isIdentifier("123var")      → returns 0 (starts with digit)
isIdentifier("var-name")    → returns 0 (contains hyphen)
```

1.5 int isNumber(char *str)

Purpose: Validates if a string represents a valid integer number.

Parameters:

- **str** - String to validate

Returns:

- `1` if the string is a valid number
- `0` if the string contains non-digit characters

Logic:

```
int isNumber(char *str)
{
    for (int i = 0; str[i] != '\0'; i++)
    {
        if (!isdigit(str[i]))
        {
            return 0; // Non-digit found
        }
    }
    return 1; // All characters are digits
}
```

How it works:

1. Iterates through each character in the string
2. Uses `isdigit()` to check if character is 0-9
3. Returns `0` immediately if non-digit found
4. Returns `1` if all characters are digits

Examples:

```
isNumber("123")    → returns 1 (valid)
isNumber("0")       → returns 1 (valid)
isNumber("12.3")    → returns 0 (contains dot)
isNumber("-5")      → returns 0 (contains minus)
isNumber("12a")     → returns 0 (contains letter)
```

2. Tokenization Functions

2.1 `void tokenize(FILE *file)`

Purpose: Reads source code from file and converts it into tokens.

Parameters:

- `file` - File pointer to source code file

Returns: None (void), but populates global `tokens[]` array

Logic:

```
void tokenize(FILE *file)
{
    char ch;
    char buffer[TOKEN_VALUE_LENGTH];
    int bufferIndex = 0;

    while ((ch = fgetc(file)) != EOF && tokenCount < NUMBER_OF_TOKENS)
    {
        // Skip whitespace
        if (isspace(ch))
            continue;

        // Handle identifiers/keywords
        if (isalpha(ch) || ch == '_') { ... }

        // Handle numbers
        else if (isdigit(ch)) { ... }

        // Handle operators and symbols
        else { ... }
    }
}
```

How it works:

Step 1: Character-by-character reading

- Uses `fgetc()` to read one character at a time
- Continues until EOF or max tokens reached

Step 2: Whitespace handling

```
if (isspace(ch))
    continue;
```

- Skips spaces, tabs, newlines

Step 3: Identifier/Keyword recognition

```
if (isalpha(ch) || ch == '_')
{
    bufferIndex = 0;
    buffer[bufferIndex++] = ch;

    // Read remaining alphanumeric characters
    while (isalpha((ch = fgetc(file))) || isdigit(ch) || ch == '_')
        buffer[bufferIndex++] = ch;

    buffer[bufferIndex] = '\0';
}
```

```
    ungetc(ch, file); // Put back non-alphanumeric character

    // Classify as keyword or identifier
    if (isKeyword(buffer))
        strcpy(tokens[tokenCount].type, "KEYWORD");
    else
        strcpy(tokens[tokenCount].type, "IDENTIFIER");

    strcpy(tokens[tokenCount].value, buffer);
    tokenCount++;
}
```

Key points:

- Reads complete word by consuming all alphanumeric characters
- Uses `ungetc()` to put back the terminating character
- Classifies as KEYWORD or IDENTIFIER

Step 4: Number recognition

```
else if (isdigit(ch))
{
    bufferIndex = 0;
    buffer[bufferIndex++] = ch;

    // Read all consecutive digits
    while (isdigit((ch = fgetc(file))))
        buffer[bufferIndex++] = ch;

    buffer[bufferIndex] = '\0';
    ungetc(ch, file);

    strcpy(tokens[tokenCount].type, "NUMBER");
    strcpy(tokens[tokenCount].value, buffer);
    tokenCount++;
}
```

Step 5: Operator/Symbol recognition

```
else
{
    buffer[0] = ch;
    buffer[1] = '\0';

    if (isOperator(buffer))
        strcpy(tokens[tokenCount].type, "OPERATOR");
    else if (isSymbol(buffer))
        strcpy(tokens[tokenCount].type, "SYMBOL");
    else
```

```
{
    printf("Error: Unrecognized token '%s'\n", buffer);
    exit(1);
}

strcpy(tokens[tokenCount].value, buffer);
tokenCount++;
}
```

Example:

```
// Input: int x = 5;
// After tokenize():
tokens[0] = {type: "KEYWORD",    value: "int"}
tokens[1] = {type: "IDENTIFIER", value: "x"}
tokens[2] = {type: "OPERATOR",   value: "="}
tokens[3] = {type: "NUMBER",     value: "5"}
tokens[4] = {type: "SYMBOL",     value: ";"}
tokenCount = 5
```

2.2 void printTokens()

Purpose: Displays all generated tokens for debugging/verification.

Parameters: None

Returns: None (void)

Logic:

```
void printTokens()
{
    for (int i = 0; i < tokenCount; i++)
    {
        printf("Token %d: Type = %s, Value = %s\n",
               i, tokens[i].type, tokens[i].value);
    }
}
```

How it works:

1. Iterates through all tokens (0 to tokenCount-1)
2. Prints token index, type, and value
3. Used for debugging and verification

Example Output:

```
Token 0: Type = KEYWORD, Value = int
Token 1: Type = IDENTIFIER, Value = y
Token 2: Type = OPERATOR, Value = =
Token 3: Type = NUMBER, Value = 5
Token 4: Type = SYMBOL, Value = ;
```

3. Symbol Table Functions

3.1 `void addVariable(char *name, int value, int isInitialized)`

Purpose: Adds a new variable to the symbol table with duplicate checking.

Parameters:

- `name` - Variable name
- `value` - Initial value (or 0 if uninitialized)
- `isInitialized` - Flag: 1 if initialized, 0 if not

Returns: None (void), but exits on error

Logic:

```
void addVariable(char *name, int value, int isInitialized)
{
    if (variableCount < MAX_VARIABLES)
    {
        // Check for duplicate declaration
        for (int i = 0; i < variableCount; i++)
        {
            if (strcmp(variables[i].name, name) == 0)
            {
                printf("Error: Variable '%s' already declared\n", name);
                exit(1);
            }
        }

        // Add new variable
        strncpy(variables[variableCount].name, name, VARIABLE_NAME_LENGTH -
1);
        variables[variableCount].name[VARIABLE_NAME_LENGTH - 1] = '\0';
        variables[variableCount].value = value;
        variables[variableCount].isInitialized = isInitialized;
        variableCount++;
    }
    else
    {
        printf("Error: Maximum variable limit reached\n");
        exit(1);
    }
}
```

How it works:

Step 1: Check capacity

- Ensures we haven't exceeded MAX_VARIABLES (100)

Step 2: Check for duplicates

- Loops through existing variables
- Compares names using `strcmp()`
- Exits with error if variable already exists

Step 3: Add variable

- Safely copies name using `strncpy()`
- Sets value and initialization flag
- Increments `variableCount`

Examples:

```
// Initialized variable
addVariable("x", 5, 1);
// variables[0] = {name: "x", value: 5, isInitialized: 1}

// Uninitialized variable
addVariable("y", 0, 0);
// variables[1] = {name: "y", value: 0, isInitialized: 0}

// Duplicate - causes error
addVariable("x", 10, 1);
// Output: Error: Variable 'x' already declared
```

3.2 `void setVariableValue(char *name, int value)`

Purpose: Updates the value of an existing variable (used in assignments).

Parameters:

- `name` - Variable name to update
- `value` - New value to assign

Returns: None (void), but exits on error

Logic:

```
void setVariableValue(char *name, int value)
{
    for (int i = 0; i < variableCount; i++)
```

```
{
    if (strcmp(variables[i].name, name) == 0)
    {
        variables[i].value = value;
        variables[i].isInitialized = 1; // Mark as initialized
        return;
    }
}
printf("Semmentic Error: Variable '%s' not found\n", name);
exit(1);
}
```

How it works:

Step 1: Search for variable

- Loops through symbol table
- Compares names using `strcmp()`

Step 2: Update value

- Sets new value
- Marks variable as initialized (important for previously uninitialized variables)
- Returns immediately after update

Step 3: Error if not found

- If loop completes without finding variable, reports semantic error
- Exits program

Examples:

```
// Assume: variables[0] = {name: "x", value: 5, isInitialized: 1}

setVariableValue("x", 10);
// Now: variables[0] = {name: "x", value: 10, isInitialized: 1}

setVariableValue("z", 20);
// Output: Semmentic Error: Variable 'z' not found
```

3.3 `int lookupVariable(char *name)`

Purpose: Retrieves the value of a variable with initialization checking.

Parameters:

- `name` - Variable name to look up

Returns:

- Variable value (int) if found and initialized
- Exits program if not found or not initialized

Logic:

```
int lookupVariable(char *name)
{
    for (int i = 0; i < variableCount; i++)
    {
        if (strcmp(variables[i].name, name) == 0)
        {
            if (variables[i].isInitialized)
            {
                return variables[i].value;
            }
            else
            {
                printf("Error: Variable '%s' is not initialized\n", name);
                exit(1);
            }
        }
    }
    printf("Error: Variable '%s' not found\n", name);
    exit(1);
}
```

How it works:**Step 1: Search symbol table**

- Iterates through all variables
- Compares names

Step 2: Check initialization

- If found, checks `isInitialized` flag
- Returns value if initialized
- Errors if not initialized

Step 3: Error if not found

- Reports variable not found
- Exits program

Examples:

```
// Symbol table:
// variables[0] = {name: "x", value: 5, isInitialized: 1}
// variables[1] = {name: "y", value: 0, isInitialized: 0}
```

```
int val1 = lookupVariable("x");  
// Returns: 5  
  
int val2 = lookupVariable("y");  
// Output: Error: Variable 'y' is not initialized  
  
int val3 = lookupVariable("z");  
// Output: Error: Variable 'z' not found
```

4. Parsing Functions

4.1 void expect(char *type, char *value)

Purpose: Validates that the current token matches expected type and value, then advances.

Parameters:

- **type** - Expected token type (e.g., "KEYWORD", "SYMBOL")
- **value** - Expected token value (e.g., "int", ";")

Returns: None (void), but exits on mismatch

Logic:

```
void expect(char *type, char *value)  
{  
    if (currentTokenIndex < tokenCount)  
    {  
        // Check type  
        if (strcmp(tokens[currentTokenIndex].type, type) != 0)  
        {  
            printf("Syntax Error: Unexpected token %s\n",  
                tokens[currentTokenIndex].value);  
            exit(1);  
        }  
  
        // Check value  
        if (strcmp(tokens[currentTokenIndex].value, value) != 0)  
        {  
            printf("Syntax Error: Expected '%s' but found '%s'\n",  
                value, tokens[currentTokenIndex].value);  
            exit(1);  
        }  
  
        // Advance to next token  
        currentTokenIndex++;  
    }  
}
```

How it works:**Step 1: Bounds check**

- Ensures we haven't run out of tokens

Step 2: Type validation

- Compares current token type with expected type
- Errors if mismatch

Step 3: Value validation

- Compares current token value with expected value
- Errors if mismatch

Step 4: Advance

- Increments `currentTokenIndex` to move to next token

Examples:

```
// Current token: {type: "KEYWORD", value: "int"}
expect("KEYWORD", "int"); // ✓ Passes, advances to next token

// Current token: {type: "SYMBOL", value: "("}
expect("SYMBOL", ";");    // ✗ Error: Expected ';' but found '('

// Current token: {type: "IDENTIFIER", value: "x"}
expect("KEYWORD", "int"); // ✗ Error: Unexpected token x
```

4.2 `void parseProgram()`

Purpose: Entry point for parsing - validates that entire input is valid.

Parameters: None

Returns: None (void)

Logic:

```
void parseProgram()
{
    parseStatementList();

    if (currentTokenIndex < tokenCount)
    {
        printf("Syntax Error: Unexpected token '%s'\n",
               tokens[currentTokenIndex].value);
        exit(1);
    }
}
```

```
}  
}
```

How it works:

Step 1: Parse all statements

- Calls `parseStatementList()` to process all statements

Step 2: Verify completion

- After parsing, checks if all tokens consumed
- If tokens remain, reports unexpected token error
- This catches extra code after the program should end

Grammar:

```
<program> → <stmt_list>
```

Example:

```
// Valid input:  
int x = 5;  
print(x);  
// All tokens consumed ✓  
  
// Invalid input:  
int x = 5; extra tokens here  
// Error: Unexpected token 'extra' ✗
```

4.3 `void parseStatementList()`

Purpose: Parses a sequence of statements (loop until no more tokens).

Parameters: None

Returns: None (void)

Logic:

```
void parseStatementList()  
{  
    while (currentTokenIndex < tokenCount)  
    {  
        parseStatement();  
    }  
}
```

How it works:

Step 1: Loop through tokens

- Continues while tokens remain

Step 2: Parse each statement

- Calls `parseStatement()` for each statement
- Each call to `parseStatement()` consumes tokens for one complete statement

Grammar:

```
<stmt_list> → <stmt> <stmt_list> | ε
```

This is an iterative implementation of the recursive grammar rule

Example:

```
// Input tokens for:
int x = 5;    // Statement 1
int y = 10;   // Statement 2
print(x);    // Statement 3

// Execution:
parseStatement(); // Processes "int x = 5;"
parseStatement(); // Processes "int y = 10;"
parseStatement(); // Processes "print(x);"
// Loop exits when currentTokenIndex == tokenCount
```

4.4 `void parseStatement()`

Purpose: Determines statement type and delegates to appropriate parser.

Parameters: None

Returns: None (void)

Logic:

```
void parseStatement()
{
    if (strcmp(tokens[currentTokenIndex].value, "int") == 0)
    {
        parseDeclaration();
    }
}
```

```
    else if (strcmp(tokens[currentTokenIndex].type, "IDENTIFIER") == 0)
    {
        parseAssign();
    }
    else if (strcmp(tokens[currentTokenIndex].value, "print") == 0)
    {
        parsePrint();
    }
    else
    {
        printf("Syntax Error: Invalid statement\n");
        exit(1);
    }
}
```

How it works:

Decision tree based on current token:

1. If token value is "int" → Variable declaration
 - Call `parseDeclaration()`
2. If token type is "IDENTIFIER" → Assignment statement
 - Call `parseAssign()`
3. If token value is "print" → Print statement
 - Call `parsePrint()`
4. Otherwise → Syntax error
 - Invalid statement start

Grammar:

```
<stmt> → <decl_stmt> | <assign_stmt> | <print_stmt>
```

Examples:

```
// Current token: {type: "KEYWORD", value: "int"}
parseStatement();
// → calls parseDeclaration()

// Current token: {type: "IDENTIFIER", value: "x"}
parseStatement();
// → calls parseAssign()

// Current token: {type: "KEYWORD", value: "print"}
```

```
parseStatement();  
// → calls parsePrint()  
  
// Current token: {type: "NUMBER", value: "5"}  
parseStatement();  
// → Error: Invalid statement
```

4.5 void parseDeclaration()

Purpose: Parses variable declaration with optional initialization.

Parameters: None

Returns: None (void)

Logic:

```
void parseDeclaration()  
{  
    expect("KEYWORD", "int");  
  
    char varName[VARIABLE_NAME_LENGTH];  
    strncpy(varName, tokens[currentTokenIndex].value, VARIABLE_NAME_LENGTH  
- 1);  
    expect("IDENTIFIER", tokens[currentTokenIndex].value);  
  
    if (currentTokenIndex < tokenCount &&  
        strcmp(tokens[currentTokenIndex].value, "=") == 0)  
    {  
        // Declaration with initialization: int x = expr;  
        expect("OPERATOR", "=");  
        int value = parseExpression();  
        addVariable(varName, value, 1); // isInitialized = 1  
    }  
    else  
    {  
        // Declaration without initialization: int x;  
        addVariable(varName, 0, 0); // isInitialized = 0  
    }  
  
    expect("SYMBOL", ";");  
}
```

How it works:

Step 1: Expect "int" keyword

```
expect("KEYWORD", "int");
```

Step 2: Capture variable name

```
char varName[VARIABLE_NAME_LENGTH];
strncpy(varName, tokens[currentTokenIndex].value, ...);
expect("IDENTIFIER", tokens[currentTokenIndex].value);
```

- Saves variable name before advancing
- Validates it's an identifier

Step 3: Check for initialization

```
if (currentTokenIndex < tokenCount &&
    strcmp(tokens[currentTokenIndex].value, "=") == 0)
```

- Looks ahead for "=" operator
- Doesn't consume token yet (just checks)

Step 4a: With initialization

```
expect("OPERATOR", "=");
int value = parseExpression();
addVariable(varName, value, 1);
```

- Consumes "=" operator
- Evaluates expression
- Adds variable with initial value and isInitialized=1

Step 4b: Without initialization

```
addVariable(varName, 0, 0);
```

- Adds variable with value=0 and isInitialized=0

Step 5: Expect semicolon

```
expect("SYMBOL", ";");
```

Grammar:

```
<decl_stmt> → "int" <id> "=" <expr> ";"
            | "int" <id> ";"
```

Examples:

```
// Input: int x = 5;
// Execution:
expect("KEYWORD", "int");      // Consumes "int"
varName = "x"
expect("IDENTIFIER", "x");      // Consumes "x"
// Sees "=", so:
expect("OPERATOR", "=");        // Consumes "="
value = parseExpression();      // Returns 5
addVariable("x", 5, 1);         // Adds to symbol table
expect("SYMBOL", ";");          // Consumes ";"

// Input: int y;
// Execution:
expect("KEYWORD", "int");      // Consumes "int"
varName = "y"
expect("IDENTIFIER", "y");      // Consumes "y"
// No "=", so:
addVariable("y", 0, 0);         // Uninitialized
expect("SYMBOL", ";");          // Consumes ";"
```

4.6 void parseAssign()**Purpose:** Parses assignment statement (updates existing variable).**Parameters:** None**Returns:** None (void)**Logic:**

```
void parseAssign()
{
    char varName[VARIABLE_NAME_LENGTH];
    strncpy(varName, tokens[currentTokenIndex].value, VARIABLE_NAME_LENGTH
- 1);

    expect("IDENTIFIER", tokens[currentTokenIndex].value);
    expect("OPERATOR", "=");
    int value = parseExpression();
    setVariableValue(varName, value);
    expect("SYMBOL", ";");
}
```

How it works:

Step 1: Capture variable name

```
char varName[VARIABLE_NAME_LENGTH];  
strncpy(varName, tokens[currentTokenIndex].value, ...);
```

- Saves identifier name before advancing token

Step 2: Expect identifier

```
expect("IDENTIFIER", tokens[currentTokenIndex].value);
```

- Validates and consumes identifier token

Step 3: Expect assignment operator

```
expect("OPERATOR", "=");
```

- Validates and consumes "="

Step 4: Evaluate expression

```
int value = parseExpression();
```

- Parses right-hand side
- Returns computed value

Step 5: Update variable

```
setVariableValue(varName, value);
```

- Updates variable in symbol table
- Checks if variable exists
- Marks as initialized

Step 6: Expect semicolon

```
expect("SYMBOL", ";");
```

Grammar:

```
<assign_stmt> → <id> "=" <expr> ";"
```

Example:

```
// Input: x = 10;
// Execution:
varName = "x" // Save name
expect("IDENTIFIER", "x"); // Consumes "x"
expect("OPERATOR", "="); // Consumes "="
value = parseExpression(); // Returns 10
setVariableValue("x", 10); // Updates x in symbol table
expect("SYMBOL", ";"); // Consumes ";"

// Symbol table before: {name: "x", value: 5, isInitialized: 1}
// Symbol table after: {name: "x", value: 10, isInitialized: 1}
```

4.7 void parsePrint()

Purpose: Parses print statement and outputs variable value.

Parameters: None

Returns: None (void)

Logic:

```
void parsePrint()
{
    expect("KEYWORD", "print");
    expect("SYMBOL", "(");

    char varName[VARIABLE_NAME_LENGTH];
    strncpy(varName, tokens[currentTokenIndex].value, VARIABLE_NAME_LENGTH
- 1);

    expect("IDENTIFIER", tokens[currentTokenIndex].value);
    expect("SYMBOL", ")");
    expect("SYMBOL", ";");

    printf("%d\n", lookupVariable(varName));
}
```

How it works:

Step 1: Expect "print" keyword

```
expect("KEYWORD", "print");
```

Step 2: Expect opening parenthesis

```
expect("SYMBOL", "(");
```

Step 3: Capture variable name

```
char varName[VARIABLE_NAME_LENGTH];  
strncpy(varName, tokens[currentTokenIndex].value, ...);
```

Step 4: Expect identifier

```
expect("IDENTIFIER", tokens[currentTokenIndex].value);
```

Step 5: Expect closing parenthesis

```
expect("SYMBOL", ");
```

Step 6: Expect semicolon

```
expect("SYMBOL", ";");
```

Step 7: Output value

```
printf("%d\n", lookupVariable(varName));
```

- Looks up variable value
- Checks if initialized
- Prints value to stdout

Grammar:

```
<print_stmt> → "print" "(" <id> ")" ";"
```

Example:

```
// Input: print(x);
// Symbol table: {name: "x", value: 25, isInitialized: 1}

// Execution:
expect("KEYWORD", "print");    // Consumes "print"
expect("SYMBOL", "(");         // Consumes "("
varName = "x"                  // Save name
expect("IDENTIFIER", "x");     // Consumes "x"
expect("SYMBOL", ")");         // Consumes ")"
expect("SYMBOL", ";");         // Consumes ";"
lookupVariable("x")            // Returns 25
printf("%d\n", 25);           // Outputs: 25
```

4.8 int parseExpression()

Purpose: Parses and evaluates arithmetic expressions (supports addition).

Parameters: None

Returns:

- **int** - The computed value of the expression

Logic:

```
int parseExpression()
{
    int leftValue;

    // Parse left operand
    if (strcmp(tokens[currentTokenIndex].type, "NUMBER") == 0)
    {
        leftValue = atoi(tokens[currentTokenIndex].value);
        expect("NUMBER", tokens[currentTokenIndex].value);
    }
    else if (strcmp(tokens[currentTokenIndex].type, "IDENTIFIER") == 0)
    {
        leftValue = lookupVariable(tokens[currentTokenIndex].value);
        expect("IDENTIFIER", tokens[currentTokenIndex].value);
    }
    else
    {
        printf("Syntax Error: Expected number or identifier\n");
        exit(1);
    }

    // Check for addition operator
    if (currentTokenIndex < tokenCount &&
        strcmp(tokens[currentTokenIndex].value, "+") == 0)
    {
```

```
        expect("OPERATOR", "+");
        int rightValue = parseExpression(); // Recursive call
        return leftValue + rightValue;
    }

    return leftValue;
}
```

How it works:

Step 1: Parse left operand

Case A: Number literal

```
if (strcmp(tokens[currentTokenIndex].type, "NUMBER") == 0)
{
    leftValue = atoi(tokens[currentTokenIndex].value);
    expect("NUMBER", tokens[currentTokenIndex].value);
}
```

- Converts string to integer using `atoi()`
- Advances token

Case B: Variable reference

```
else if (strcmp(tokens[currentTokenIndex].type, "IDENTIFIER") == 0)
{
    leftValue = lookupVariable(tokens[currentTokenIndex].value);
    expect("IDENTIFIER", tokens[currentTokenIndex].value);
}
```

- Looks up variable value from symbol table
- Validates variable exists and is initialized
- Advances token

Case C: Error

```
else
{
    printf("Syntax Error: Expected number or identifier\n");
    exit(1);
}
```

Step 2: Check for addition

```

if (currentTokenIndex < tokenCount &&
    strcmp(tokens[currentTokenIndex].value, "+") == 0)
{
    expect("OPERATOR", "+");
    int rightValue = parseExpression(); // RECURSIVE CALL
    return leftValue + rightValue;
}

```

- Looks ahead for "+" operator
- If found, recursively parses right side
- Adds values and returns result

Step 3: Return single value

```

return leftValue;

```

- If no operator, returns just the left value

Grammar:

```

<expr> → <term> "+" <expr> | <term>
<term> → <id> | <number>

```

Recursion enables right-associativity and chained additions

Examples:

Example 1: Simple number

```

// Input: 5
leftValue = 5 // Parse "5"
// No "+", so:
return 5

```

Example 2: Simple addition

```

// Input: 5 + 10
leftValue = 5 // Parse "5"
// See "+", so:
expect("+") // Consume "+"
rightValue = parseExpression()
    leftValue = 10 // Parse "10"
    return 10
return 5 + 10 = 15

```

Example 3: Variable reference

```
// Input: x + y
// Symbol table: x=5, y=10

leftValue = lookupVariable("x") // Returns 5
expect("IDENTIFIER", "x")
// See "+", so:
expect("+")
rightValue = parseExpression()
  leftValue = lookupVariable("y") // Returns 10
  expect("IDENTIFIER", "y")
  return 10
return 5 + 10 = 15
```

Example 4: Chained addition (right-associative)

```
// Input: 5 + 10 + 15

leftValue = 5 // Parse "5"
// See "+", so:
expect("+")
rightValue = parseExpression() // Parse "10 + 15"
  leftValue = 10 // Parse "10"
  // See "+", so:
  expect("+")
  rightValue = parseExpression() // Parse "15"
  leftValue = 15 // Parse "15"
  return 15
  return 10 + 15 = 25
return 5 + 25 = 30
```

Call stack visualization:

```
parseExpression("5 + 10 + 15")
├─ leftValue = 5
└─ parseExpression("10 + 15")
    ├─ leftValue = 10
    └─ parseExpression("15")
        └─ return 15
    └─ return 10 + 15 = 25
└─ return 5 + 25 = 30
```

5. Main Function

5.1 `int main(int argc, char *argv[])`

Purpose: Program entry point - orchestrates tokenization, parsing, and execution.

Parameters:

- `argc` - Argument count
- `argv[]` - Argument values (`argv[1]` should be input filename)

Returns:

- `0` on success
- `1` on file error

Logic:

```
int main(int argc, char *argv[])
{
    FILE *file;
    char line[LINE_LENGTH];

    file = fopen(argv[1], "r");

    if (file == NULL)
    {
        perror("Error opening file");
        return 1;
    }

    tokenize(file);
    printTokens();
    parseProgram();

    fclose(file);
    return 0;
}
```

How it works:

Step 1: Open input file

```
file = fopen(argv[1], "r");
```

- Opens file specified as command-line argument
- Mode "r" = read-only

Step 2: Error checking

```
if (file == NULL)
{
    perror("Error opening file");
    return 1;
}
```

- Checks if file opened successfully
- Prints error message if failed
- Returns non-zero exit code

Step 3: Tokenization

```
tokenize(file);
```

- Reads source code and converts to tokens
- Populates global `tokens[]` array

Step 4: Display tokens (debugging)

```
printTokens();
```

- Prints all tokens for verification

Step 5: Parse and execute

```
parseProgram();
```

- Validates syntax
- Executes code (single-pass interpreter)
- Print statements output during parsing

Step 6: Cleanup

```
fclose(file);
return 0;
```

- Closes file
- Returns success code

Example Execution:

```
$ ./parser input.txt
```

Where input.txt contains:

```
int y = 5;
int x = 20;
int z = x + y;
print(z);
```

Output:

```
Token 0: Type = KEYWORD, Value = int
Token 1: Type = IDENTIFIER, Value = y
Token 2: Type = OPERATOR, Value = =
Token 3: Type = NUMBER, Value = 5
Token 4: Type = SYMBOL, Value = ;
[... more tokens ...]
25
```

Summary of Function Call Flow

```
main()
├─ fopen()
├─ tokenize()
│   ├── isKeyword()
│   ├── isIdentifier()
│   ├── isNumber()
│   ├── isOperator()
│   └─ isSymbol()
├─ printTokens()
├─ parseProgram()
│   └─ parseStatementList()
│       └─ parseStatement() [called multiple times]
│           ├── parseDeclaration()
│           │   ├── expect()
│           │   ├── parseExpression()
│           │   │   ├── lookupVariable()
│           │   │   └─ parseExpression() [recursive]
│           │   └─ addVariable()
│           ├── parseAssign()
│           │   ├── expect()
│           │   ├── parseExpression()
│           │   └─ setVariableValue()
│           └─ parsePrint()
│               ├── expect()
│               └─ lookupVariable()
└─ fclose()
```

Key Design Patterns

1. Recursive Descent Parsing

Each grammar rule has a corresponding function.

2. Single-Pass Compilation

Tokenize → Parse → Execute all in one pass.

3. Symbol Table

Centralized variable management with semantic checking.

4. Lookahead

Functions peek at next token without consuming it.

5. Error Handling

Immediate exit on syntax/semantic errors with descriptive messages.

End of Function-Wise Explanation